

How the free/paid AI split actually works

A plain-language walkthrough — updated July 5, 2026 with a corrected capacity model (a real error was caught and fixed) plus real measured numbers

Correction notice: An earlier version of this document mislabeled OpenAI's "Batch queue limit" (a separate, unrelated limit for OpenAI's asynchronous Batch API) as if it were a real-time daily token cap for normal chat usage. That mistake understated Tier 1's real capacity by roughly 4x. This version is corrected: the real constraint is Tier 1's actual daily request cap (10,000 RPD) combined with how many API calls one full loop takes (6). The corrected story is better news than before, not worse.

1. What is a "loop"?

A loop is one full trip through the app for one moment — start to finish: capture, 3 reflection turns, prayer, journal. We ran this exact loop live 4 times to find a reliable number.

Run	Tokens	Notes
Run 1 (original prompt)	6,708	Before the wrap-up-to-prayer refinement — treated as a pre-optimization outlier, not part of the benchmark
Run 2 (refined prompt)	4,802	Benchmark run
Run 3 (refined prompt)	4,691	Benchmark run
Run 4 (refined prompt)	4,622	Benchmark run

Runs 2, 3, and 4 all used the refined prompt (the one that has the assistant invite prayer itself right after the 3rd reflection turn, instead of a separate call). They landed within 4% of each other — **4,802, 4,691, 4,622 — a genuinely stable,**

repeatable number, not noise. Average: 4,705 tokens per loop, across 6 real API calls. This is the benchmark used throughout this document.

Every one of these numbers is confirmed twice — once from OpenAI's own usage API, once from summing the real per-call numbers straight from the transcripts. Real cost split: ~4,402 input tokens + ~303 output tokens per loop, on average.

2. Two AI companies can answer those asks: Groq (free) and OpenAI's GPT-4o mini (paid)

Every call in the loop gets routed to one or the other. Here's the real difference between them, using the 4,705-token benchmark loop and 6 calls/loop:

Groq — free

\$0

up to about 21 loops per day, combined across every user. After that, Groq simply stops answering until the next day. This is a hard wall, not negotiable, because it's free.

GPT-4o mini — paid

\$0.00084

per loop, every loop. No fixed daily loop limit beyond the account's current rate limits and monthly spend/budget limits. Corrected real ceiling: about 1,666 loops per day (10,000 requests/day ÷ 6 calls per loop), and about 42 loops per minute. Past that rate limit, extra people are queued/retried by our own app, or may receive a temporary rate-limit error if we don't handle retries ourselves — OpenAI doesn't automatically queue politely for us.

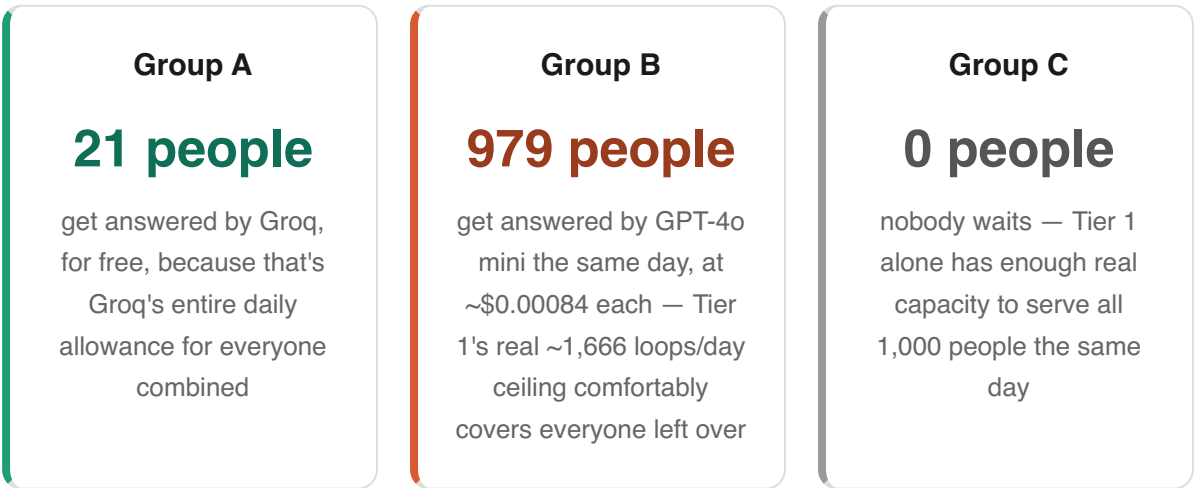
Confirmed real pricing, straight from OpenAI: GPT-4o mini costs exactly **\$0.00000015 per input token** and **\$0.00000060 per output token** — the same as \$0.15 and \$0.60 per million tokens. This matches our own billing data exactly (e.g. \$0.003109 charged for 20,725 input tokens = \$0.00015/token, precisely). Not an estimate — the real, published rate.

3. What "21 loops a day" and "1,666 loops a day" actually mean

	Groq (free)	GPT-4o mini (paid)
Is there a total limit on how much you can ever use?	Yes — resets daily, but capped every day	No fixed daily cap at Tier 2+, but usage is still bounded by rate limits, monthly spend caps, and configured budgets
What happens if too many people ask at once?	Extra people get nothing until tomorrow	Extra people are queued/retried by our app, or get a temporary rate-limit error if we don't handle retries ourselves
Cost	\$0	~\$0.00084 per loop
Real daily ceiling (corrected)	~21 loops/day, shared	~1,666 loops/day (10,000 RPD ÷ 6 calls/loop)

4. If 1,000 people sign up and do their first loop on the same day

Here's the corrected split — and it's much better news than the earlier version of this document said.



21 + 979 + 0 = 1,000 people. Everyone served, same day, on Tier 1 alone.



■ Group A, 21 people (too thin to show a label, but it's there on the far left)

The earlier version of this document said 687 (then 554) people would be stuck waiting. That was wrong — it came from mistakenly using OpenAI's Batch API queue limit as if it were a real-time daily cap. The real constraint (10,000 requests/day) is far more generous once you divide by the actual 6 calls per loop.

\$0 cost for the 21 free loops	\$0.82 cost for the 979 paid loops	~\$0.82 total, to get all 1,000 people through their first loop, same day
--	--	---

5. Real Tier 1-5 rate limits for GPT-4o mini (published by OpenAI)

Tier	Total ever paid			Monthly spend cap
Tier 1 (where we are today)	\$5			\$100/month
Tier 2 (what \$50 unlocks)	\$50			\$500/month

Tier	RPM	RPD	TPM	Batch queue limit
Free	—	—	—	Not supported
Tier 1 (today)	500	10,000	200,000	2,000,000
Tier 2 (\$50 paid)	5,000	—	2,000,000	20,000,000
Tier 3	5,000	—	4,000,000	40,000,000
Tier 4	10,000	—	10,000,000	1,000,000,000
Tier 5	30,000	—	150,000,000	15,000,000,000

Batch queue limit ≠ real-time daily cap. That column governs OpenAI's separate, asynchronous Batch API — pending tokens queued for batch processing, not same-day chat traffic. Real-time capacity comes from RPM/RPD/TPM only. Note RPD disappears (shown as "—") from Tier 2 onward — no daily request cap at all beyond Tier 1.

Since Tier 1 alone already serves all 1,000 people same-day, what does Tier 2 actually buy you? Headroom for real scale beyond ~1,666 loops/day combined demand — not relief for this specific scenario.

Tier 1 real ceiling		Tier 2 real ceiling	
Loops/day	~1,666	Loops/day	no daily cap — bounded by RPM/TPM instead
Loops/minute	~42	Loops/minute	~425 (10x Tier 1)

6. Worst case: what if someone actually maxed out the daily limit?

A natural follow-up question: how much could Tier 1 or Tier 2 cost on the single worst day, if something (a bug, an attack, a runaway loop) tried to use the absolute maximum allowed? The corrected math here is a real risk to plan around — these worst-case numbers are meaningfully higher than we first thought.

Scenario	Math	Approx. cost/day
Tier 1, all-input flood (via TPM)	200,000 TPM × 1,440 min = 288M input tokens	\$43.20
Tier 1, all-output flood (via RPD × max output)	10,000 requests/day × 16,384 max output tokens/response = 163.84M output tokens	\$98.30
Tier 2, all-input flood (via TPM)	2,000,000 TPM × 1,440 min = 2.88B input tokens	\$432.00
Tier 2, all-output flood (via TPM)	2,000,000 TPM × 1,440 min = 2.88B output tokens	\$1,728.00

Tier 2's worst case (\$1,728) actually exceeds its own \$500/month spend cap in a single day — meaning the monthly budget cap, not the rate limit, is the real backstop there, and it shouldn't be relied on alone.

Tier 1	Tier 2+
--------	---------

What stops runaway usage?	Hits the 10,000 requests/day wall — blocked automatically, worst case ~\$43-98/day	Nothing, until the \$500/month spend cap is hit — worst case could reach \$1,728/day theoretically
---------------------------	--	--

The real takeaway isn't a dollar figure — it's a missing safety net. At Tier 1, a runaway bug is naturally bounded to well under \$100/day by the request cap itself. At Tier 2, that automatic brake disappears — the only thing standing between normal usage and burning the full \$500/month budget in one afternoon is whatever budget alerts or hard spend limits we configure ourselves. Worth setting up before ever moving to Tier 2, not after.

Recommended safety controls before public launch or any Tier 2 move:

Control	Starting point	Why
Project budget	\$10-\$25/month during testing	Low blast radius while validating real usage
Per-user loop limit	e.g. 1 free loop/day; 3-5/day logged in	Stops one account from running up the bill
Per-IP throttling	Rate-limit unauthenticated traffic	Helps against bots/anonymous abuse
Max output tokens	300-800 tokens for normal responses	Output costs 4x input — cap long replies
Conversation/session cap	Stop after defined loop steps	Prevents runaway reflection chains
Server-side model calls only	Never expose the API key client-side	Prevents key theft and uncontrolled requests
Usage logging	Log user_id, request_id, model, tokens, cost	Enables debugging and cost attribution
Budget alerts	Alert at \$1, \$5, \$10, \$25	Early warning before budget exhaustion

7. The bottom line

Getting all 1,000 first-time users all the way through capture, 3 reflection turns, a prayer, and a journal entry costs about **\$0.82**, total — and, corrected, **every single one of them can be served the same day on Tier 1 alone**, no Tier 2 upgrade needed for this scenario. The free Groq tier covers a small slice of that for nothing; the rest costs less than a tenth of a cent per person on GPT-4o mini. The 4,705-token benchmark is backed by three consistent, repeated live tests. The one real risk worth planning for isn't normal usage — it's uncontrolled/runaway usage, which is why the safety controls above (per-user limits, output caps, budget alerts) matter before scaling past Tier 1.